PROJECT SPECIFICATION

AI Decision Copilot for Customer Retention

Local ML-Based Decision-Support Tool with LLM Q&A Interface

1. Business Problem Statement

Customer churn — the loss of customers to competitors or disengagement — is a persistent, high-cost challenge across subscription-based industries including telecommunications, SaaS, e-commerce, and financial services. Research consistently shows that retaining an existing customer costs significantly less than acquiring a new one does, yet most organizations lack tools that help frontline analysts make fast, explainable retention decisions.

The specific decision this project targets is:

> *"Should this customer be flagged as high churn risk, and what action should be taken to retain them?"*

This decision is recurring (made daily by customer success teams), high-impact (directly tied to revenue), and currently under-supported by tools that combine data-driven predictive modeling with explainable, actionable recommendations.

Stakeholders:

- Customer Success Managers — primary users who monitor account health and renewal likelihood
- Business Analysts — secondary users who analyze churn trends across segments
- Marketing Teams — consumers of recommendations for targeted retention campaigns
- Product Managers — stakeholders who use churn signals to inform product decisions

---

2. Tool Concept and Scope

2.1 Primary Job-To-Be-Done

"This tool exists to help a business analyst predict customer churn risk and interactively explore results by combining a predictive machine learning model with a local LLM-powered Q&A agent."

2.2 Scope Variant

Selected: Two-Layer Architecture (ML Prediction + LLM Q&A)

The tool operates in two distinct layers:

Layer 1 uses an XGBoost or LightGBM model to predict churn probability for all customers and generate SHAP-based explanations.

Layer 2 provides a conversational Q&A interface where the user can ask free-form questions about results and data. This balances prediction accuracy with interactive usability while

remaining feasible within hardware and timeline constraints.

2.3 Core User Stories

1. As a business analyst, I want to input a customer ID and receive a churn risk classification so that I can quickly identify whether the customer requires intervention.
2. As a business analyst, I want the system to explain the key factors contributing to churn risk so that I can understand the reasoning behind the classification.
3. As a business analyst, I want to ask free-form questions about prediction results so that I can interactively explore insights without writing code.
4. As a business analyst, I want to view the top N high-risk customers from a dataset so that I can prioritize outreach efforts effectively.
5. As a business analyst, I want consistent and structured outputs for similar inputs so that I can rely on the system in a decision-support context.

2.4 Out-of-Scope

The following are explicitly excluded from this project:

- Real-time data ingestion or integration with live enterprise CRM or data warehouse systems
- Web-based, desktop GUI, or full-stack application interfaces (CLI-based interaction only)
- Large-scale batch processing with repeated LLM calls across entire datasets without limits
- Multi-agent orchestration or distributed agent architectures
- Retrieval-Augmented Generation (RAG) pipelines with external knowledge bases
- Cloud deployment, cloud inference, or production-grade infrastructure
- Automated email or CRM action execution based on recommendations

---

3. Architecture Design Decisions

3.1 Local LLM Model

Decision	Choice
Model	Phi-3 Mini (3.8B parameters, quantized)
Runtime	Ollama (primary) / LM Studio (fallback)
Rationale	Used exclusively as a Q&A and interpretation agent. Not used for churn prediction. Runs locally on CPU with low memory footprint.
Inference Parameters	temperature = 0 for deterministic outputs; structured prompt templates

3.2 System Design Pattern

Layer 1 — Predictive ML Model (XGBoost or LightGBM) Handles all churn prediction tasks. Trained on the Telco dataset. Uses SHAP values to quantify feature-level contributions for each prediction, enabling local interpretability of individual customer churn risk. This layer produces consistent and reproducible predictions given a fixed trained model and controlled random seed. It does not involve the LLM.

Layer 2 — Local LLM Q&A Agent (Phi-3 Mini via Ollama) Handles all user interaction. The user can ask free-form questions about prediction results, customer data, and exploratory analysis. The LLM receives only structured outputs from Layer 1 (predictions, SHAP values, and aggregated statistics) and generates responses strictly grounded in this data. It does not make predictions — it only interprets and explains.

Data Flow:
CSV Data → Preprocessing → ML Model (XGBoost/LightGBM) → Predictions + SHAP Explanations → Structured Output Layer → LLM Q&A Agent → User Interaction

Design Rationale (ML Prediction + LLM Interpretation Layer)

The interactive behavior is provided by the LLM Q&A layer:

- Tool 1 — Data Loader: Loads and preprocesses the Telco CSV dataset.
- Tool 2 — ML Prediction Engine (XGBoost/LightGBM): Generates churn predictions and SHAP-based feature importance scores for each customer.
- Tool 3 — LLM Q&A Agent (Phi-3 Mini): Receives user questions, retrieves relevant structured data (predictions, SHAP values, and summary statistics) and generates clear natural language responses.

The LLM retrieves relevant structured data based on the user's query and generates grounded natural language responses, enabling interactive exploration of model outputs.

3.3 Memory

Selected: Limited Session Context

The system maintains limited session context within a single interaction to support short follow-up queries. Each session is independent, and no long-term memory is stored beyond the active session.

3.4 Interaction Style

Selected: Command-Line Interface (CLI)

A CLI interface allows full focus on core analytics and LLM reasoning components without UI development overhead. It also makes the pipeline transparent and easy to evaluate during testing.

4. Functional Requirements

The following statements define the required behaviors of the system. Each statement is independently verifiable.

1. The system shall accept a CSV file containing customer behavioral and demographic data as input.
2. The system shall validate the input dataset against a predefined schema and reject invalid or incomplete files with a clear error message.
3. The system shall parse raw customer records into a structured internal representation for processing.
4. The system shall preprocess missing, null, or inconsistent values using predefined deterministic rules.
5. The system shall train an XGBoost or LightGBM model during an initial training phase and use the trained model to predict customer churn probability.
6. The system shall split the dataset into training and test sets and report model performance metrics including accuracy, AUC, precision, and recall.
7. The system shall use SHAP values to identify the top contributing features for each individual customer prediction.
8. The system shall expose prediction results and SHAP explanations as structured data available to the LLM Q&A agent.
9. The system shall provide a conversational Q&A interface powered by a local LLM where users can ask questions about prediction results and dataset insights.
10. The system shall allow the user to ask questions such as:
    - "Why is customer X predicted to churn?"
    - "Which customers are highest risk today?"
    - "What is the churn rate for fiber optic customers?"
11. The system shall ground all LLM responses in actual data and model outputs, not general knowledge.
12. The system shall generate a --summary output that includes aggregated statistics of the dataset, including total customers processed, risk categories derived from predicted churn probability, and average MonthlyCharges per risk category.
13. The system shall define churn risk categories (Low, Medium, High) based on configurable probability thresholds (e.g., Low < 0.3, Medium 0.3–0.7, High > 0.7).
14. The system shall provide actionable retention recommendations derived from model outputs and feature contributions.

---

5. Non-Functional Requirements

5.1 Response Latency

Response latency targets will be established after the initial prototype is built and baseline measurements are taken. The system will be optimized based on observed performance rather than predetermined thresholds.

5.2 Offline Operation

The system shall operate fully offline after model initialization and shall not require internet connectivity during execution. All inference shall run locally using Ollama or LM Studio.

5.3 Privacy

- The system shall process all customer data locally on the user's machine.
- The system shall not transmit any customer data to external APIs or services.
- The system shall not persist raw customer data unless explicitly instructed by the user.

5.4 Reproducibility

ML model predictions shall be fully reproducible given the same trained model and input data, achieved by setting a fixed random seed during training. LLM responses may vary slightly in wording but shall remain consistent in meaning and grounded in the same underlying data.

5.5 Failure Modes

- The system shall validate input schema and reject files with missing required fields with a descriptive error message.
- The system shall handle missing or corrupted values using fallback imputation strategies without crashing.
- The system shall return a controlled fallback response when the LLM fails or produces invalid output after two retry attempts.
- The system shall handle an unknown customer ID gracefully by returning: "Customer ID [X] not found in dataset."

---

6. Interface Requirements

The system shall provide a command-line interface (CLI) supporting the following operations:

6.1 Single Customer Analysis

```
python main.py --file customers.csv --customer_id 101
```

Returns full churn risk analysis for a specific customer including churn probability, derived risk category, and top contributing features (SHAP-based).

6.2 Batch Processing

```
python main.py --file customers.csv --all
```

Runs the ML model on all customers in the dataset, generates churn predictions and SHAP values for all records, and saves results to outputs/predictions.csv for review.

6.3 Summary Statistics

```
python main.py --file customers.csv --summary
```

Outputs aggregated statistics: total customers processed, count of Low/Medium/High risk classifications, and average MonthlyCharges per risk category. No LLM invocation required.

6.4 Example Output (Single Customer)

```
Customer ID: 7590-VHVEG
--------------------------------------------
Churn Probability:  87%
Risk Category:  HIGH
--------------------------------------------
Top Contributing Factors (SHAP):
  1. Month-to-month contract        +0.42
  2. Short tenure (2 months)        +0.38
  3. High monthly charges ($85.70) +0.31
  4. No Online Security             +0.21
--------------------------------------------
Ask a question about this customer:
> Why is this customer at risk?

Response:
Based on the model output, this customer has an 87%
probability of churning. The strongest factors driving
this prediction are their month-to-month contract and
very short tenure of 2 months, suggesting low commitment.
High charges without protective services further increase
the risk.
--------------------------------------------
```

---

7. Data Requirements

7.1 Dataset

The system shall use the IBM Telco Customer Churn dataset sourced from Kaggle:

https://www.kaggle.com/datasets/blastchar/telco-customer-churn

The dataset contains 7,043 customer records with 21 attributes covering identity, demographics, account information, service usage, and churn status.

7.2 Key Features Used

Feature	Type	Role in ML Model
tenure	Numeric	Low tenure = higher risk
Contract	Categorical	Month-to-month = high risk
MonthlyCharges	Numeric	High charges = higher risk
TechSupport	Categorical	No support = higher risk
OnlineSecurity	Categorical	No security = higher risk
InternetService	Categorical	Fiber optic = risk signal
PaymentMethod	Categorical	Electronic check = risk signal
Churn	Binary	Ground truth for evaluation

7.3 Preprocessing Steps

- TotalCharges: Convert blank strings to numeric values; impute missing values with the column median.
- Contract: Encode using label encoding or one-hot encoding for ML model input. Month-to-month, One year, and Two year are treated as categorical values.
- SeniorCitizen: Already binary (0/1); no transformation required.
- Categorical **yes/no** fields: Standardize to lowercase for consistent rule evaluation.
- customerID: Retain as string index; exclude from scoring computation.

7.4 Local Storage Policy

- The dataset shall be stored locally in CSV format in the /data directory.
- No external database systems or cloud storage shall be used.
- Session logs shall be written to /outputs/session.log and retained locally.

---

8. Evaluation Plan

8.1 Quantitative Metrics

Performance metrics and targets will be established empirically after the initial model is trained. The following metrics will be measured and reported:
- Model AUC (Area Under the ROC Curve).
- Accuracy, Precision, and Recall on test set.
- SHAP feature importance rankings.
- LLM response relevance and grounding (evaluated qualitatively).

- System response time per query (measured, not pre-specified).

Improvement targets will be defined after baseline system behavior is observed and measured.

8.2 LLM Q&A Response Quality Rubric

Each LLM response to a user question is scored on a 0-3 scale:

Score	Criteria
3	Response directly answers the question, cites specific data or model output, contains no fabricated information
2	Response answers the question but explanation is weak or partially references incorrect data
1	Response is vague or only partially answers the question
0	Response is incorrect, hallucinated, or completely misses the question

8.3 Test Scenarios

Scenario 1 — High Risk Customer (Fully Specified)

Input:

Customer Record: customerID = 7590-VHVEG, tenure = 2, Contract = Month-to-month, MonthlyCharges = 85.70, TotalCharges = 171.40, InternetService = Fiber optic, OnlineSecurity = No, TechSupport = No, PaymentMethod = Electronic check.

Expected Output:
Churn probability above 70%, at least 3 SHAP factors identified including contract type and tenure, LLM Q&A correctly answers follow-up question about this customer.

Scenario 2 — Low Risk Customer

Input:

tenure ≥ 48 months, Contract = Two year, MonthlyCharges < 50, OnlineSecurity = Yes, TechSupport = Yes.

Expected Output:
Churn probability below 30%, SHAP factors show positive indicators such as long tenure and annual contract.

Scenario 3 — Edge Case: Missing Data

Input:

CSV with TotalCharges field blank, OnlineSecurity field null for selected customer.

Expected Output:

System applies fallback imputation, generates valid output without crashing, logs a warning to session.log.

Scenario 4 — Unknown Customer ID

Input:

--customer_id 99999 where ID does not exist in dataset.

Expected Output:

System returns controlled message “Customer ID 99999 not found in dataset.” without crashing.

Scenario 5 — Invalid CSV Schema

Input:

CSV missing the Contract column.

Expected Output:

System rejects file with error message specifying missing required field. No partial processing attempted.

Scenario 6 — Corrupted Numeric Field

Input:
MonthlyCharges = "N/A" or non-numeric value in dataset

Expected Output:

System handles invalid numeric conversion using fallback imputation strategy (e.g., median or predefined default), logs a warning, and continues processing without crashing.

---

9. Nine-Week Implementation Timeline

Week	Focus	Deliverables	Definition of Done
1	Problem Finalization + Evaluation Design	Finalized spec on GitHub, EDA notebook, 10 test cases, repo structure	Spec committed; EDA shows Churn distribution; test cases cover all risk categories.
2	Architecture + Tech Stack	Ollama + Phi-3 Mini installed, all modules scaffolded, requirements.txt	LLM responds locally within acceptable observed latency during testing; all modules have function signatures
3	ML Model Development	Train XGBoost/LightGBM model, evaluate on test set, implement SHAP	Model trains successfully, SHAP values generated for test customers,

Week	Focus	Deliverables	Definition of Done
		values, document model performance metrics	performance metrics reported
4	LLM Q&A Agent Integration	Connect Phi-3 Mini via Ollama, build Q&A interface, test with 5 sample questions	User can ask a question and receive a grounded answer based on model output and data
5	Batch Mode + Summary Command	--all flag runs ML model on full dataset and saves predictions.csv. --summary flag shows churn distribution statistics. --high-risk flag lists all customers above threshold probability.	--summary matches spec format; --all processes 100 records without error
6	Guardrails + Logging + Robustness	Input validation, fallback responses, session.log output	System handles all 5 edge cases without crashing; logs generated
7	Evaluation Execution	All 10 test cases scored; evaluation report committed	All test cases executed and results documented. LLM responses evaluated using rubric. Performance baseline established and recorded.
8	Polish + Demo Script	README.md, demo script, bug fixes from Week 7	New user can run system from README alone; demo runs without errors
9	Final Report + Postmortem	Final report (2,000+ words), complete GitHub repo, email to professor	Repo is public, organized, and complete with all deliverables

10. Skills and Technology Learning Map

Weeks 1–2 — Foundation Skills

Skill	Tool / Library	Practice Task
CSV loading and filtering	pandas	Load Telco CSV; print churn count and avg MonthlyCharges for month-to-month contracts
JSON parsing	json (stdlib)	Convert risk result dict to JSON string; parse back; print risk_category field
CLI building	argparse	Build script accepting --file and --customer_id; print confirmation message
Dependency management	venv, pip	Create clean environment; install all dependencies; export requirements.txt
Version control	Git, GitHub	Create repo with /data /src /tests /docs /outputs; commit initial scaffold

Weeks 3–4 — Core Implementation Skills

Skill	Tool / Library	Practice Task
Structured prompt engineering	Ollama API	Engineer prompts to enforce structured JSON outputs from LLM; implement validation, retry mechanisms, and parsing strategies to handle occasional malformed or non-JSON responses
JSON schema validation	json, re	Validate LLM output has required fields; implement retry on failure
LLM API integration	ollama Python library	Run Phi-3 Mini locally; verify stable response generation under observed system conditions
String parsing for LLM output	str methods, re	Extract JSON from LLM response that includes preamble text
Train churn model, evaluate with AUC	scikit-learn / XGBoost	Train XGBoost on Telco data, print AUC score on test set

Generate feature importance	SHAP	Generate SHAP values for customer 7590-VHVEG, print top 3 contributing features

Weeks 5–6 — Integration and Robustness Skills

Skill	Tool / Library	Practice Task
Batch processing	pandas itertuples, tqdm	Process 50 records; show progress bar; filter to Medium/High only
Structured logging	logging (stdlib)	Log timestamp, customer_id, prediction_confidence, response_time, llm_response_quality
Exception handling	try/except/finally	Handle missing CSV column, unknown ID, and all-null row without crashing
CLI output formatting	Python string formatting	Print aligned risk report table to terminal

Weeks 7–9 — Evaluation and Reporting Skills

Skill	Tool / Library	Practice Task
Unit testing	pytest	Write test_high_risk(), test_low_risk(), test_missing_field(); confirm all pass
Structured evaluation	Manual rubric (0–3 scale)	Score 5 LLM outputs; record in evaluation table; identify prompt improvements
Technical writing	Markdown	Write README with setup instructions usable by a new user
Final reporting	Markdown / Word	Document design decisions, results, limitations, and lessons learned (2,000+ words)